Characterizing and Predicting Scientific Workloads for Heterogeneous Computing Systems

Beau Johnston

February 2019

A thesis submitted for the degree of Doctor of Philosophy of The Australian National University.

PREVIEW

© Copyright by Beau Johnston 2019
All Rights Reserved

ProQuest Number: 27792410

All rights reserved

INFORMATION TO ALL USERS
The quality of this reproduction is dependent on the quality of the copy submitted.

In the unlikely event that the author did not send a complete manuscript
and there are missing pages, these will be noted. Also, if material had to be removed,
a note will indicate the deletion.

ProQuest 27792410

Published by ProQuest LLC (2020). Copyright of the Dissertation is held by the Author.

All Rights Reserved.
This work is protected against unauthorized copying under Title 17, United States Code
Microform Edition © ProQuest LLC.

ProQuest LLC
789 East Eisenhower Parkway
P.O. Box 1346
Ann Arbor, MI 48106 - 1346

PREVIEW

Declaration

The work in this thesis is my own except where otherwise stated.

Beau Johnston

PREVIEW

Acknowledgements

Thank you to my supervisors Josh, Greg and Alistair. Josh, you've been an excellent friend and mentor. I wouldn't have a thesis if you weren't so supportive or engaged in this work – it has actually been a lot of fun working so closely and bouncing ideas off each other. Greg, you've always supported me, technically and emotionally. I feel privileged that we've been able to collaborate so closely despite being 750 kilometers apart. Alistair, thank you for bringing me to the ANU and being so patient with this perpetual student, you have shaped me – for better or worse – into thinking like a cynical scientist. You've taught me the basics and still somehow found the time – and funding – for me.

You have all had to repeat yourselves far too frequently and still somehow maintained the vigour to keep me on track – tempering my wild ideas and pushing me through the writing up process. I wouldn't have considered spending my life as a researcher or scientist if my role-models had been any less kind or brilliant. Thank you so much!

Thank you to my loving family – Sue, Paul, Eddie, Betty and Ron – I've been incredibly lucky to come from such a supportive environment, there is no way I would have had the confidence to even enrol in university much less stick through a PhD program.

My long-time suffering partner Flo… sorry! You have been on the front-lines of work complaints, frustrations and my sheer single mindedness to get this bloody thing finished for far too long. I can't imagine how you still have the determination to start your own PhD shortly – I'm very proud of you. Also, sorry about coming home at 5am with no teeth – and the PhD-esq Stockholm syndrome that caused it.

I would also like to thank all my office buddies over the years – Brian, Edwin, Jess, Shayanti, Gaurav, Anish, Luke, Kunshan and Guyver – without the support and communal suffering you all provided I would have abandoned my PhD a long time ago. At various stages, you have all fed me, helped me de-stress by running me round a badminton court, listened to my ample complaints and discuss my – from your perspectives what must be especially boring – ideas. You have all been amazingly supportive friends and family and I love you all very much.

Thank you to my colleagues at The University of Bristol's High Performance Computing Research group for the use of `The Zoo` Research cluster which was used for the experimental evaluation, it was critical to generating the runtime results.

Finally, I would also like to thank Oracle and the ANU VC Travel Grants Office for providing additional funding, this was invaluable for conference attendance.

PREVIEW

Abstract

The next-generation of supercomputers will feature a diverse mix of accelerator devices. The increase in heterogeneity is explained by the nature of supercomputing workloads – certain devices offer acceleration, or a shorter time to completion, for particular application programs. Certain characteristics of these programs are fixed and impose fundamental limitations on the workloads regardless of which accelerator is used for the computation; for instance, a graph traversal program always exhibits the same high-branch and low-computation properties regardless of what device is used to execute it. To support efficient scheduling on High Performance Computing (HPC) systems it is necessary to make accurate performance predictions for workloads on varied compute devices, which is challenging due to diverse computation, communication and memory access characteristics which result in varying performance between devices. On HPC systems a single node may feature a Graphics Processing Unit (GPU), a Central Processing Unit (CPU), and a Field-Programmable Gate Array (FPGA) or Many Integrated Core (MIC) device. This work presents a device independent predictor – a methodology to use device-independent characteristics of scientific codes to select the optimal accelerator device with regard to execution time or energy expenditure.

Open Compute Language (OpenCL) is a programming model designed to facilitate the development of application codes capable of running on multiple different devices. First released in late 2008, it defines a C-like language used to write kernels that can be compiled to run on the different devices. Implementations of the current release (2.2) exist for CPUs, GPUs, FPGAs and the Intel MIC systems, and as such, there is increasing interest in the use of OpenCL for developing scientific applications designed to run on next-generation supercomputer systems.

This thesis seeks to use the device-independent characteristics of an OpenCL code to select the optimal accelerator device on which to execute each OpenCL kernel. Consideration is given both to execution time and energy usage.

The first focus of this work is to present a comprehensive benchmark suite for OpenCL in the heterogeneous HPC setting: an extended and enhanced version of the OpenDwarfs OpenCL benchmark suite. Our extensions improve the portability and robustness of the applications, the correctness of results and the choice of problem size, and diversity of coverage through the inclusion of additional application patterns. This work manifests in performance measurements on a set 15 devices and over 12 applications.

We next present our Architecture Independent Workload Characterization (AIWC) tool which characterizes OpenCL kernels according to a set of architecture-independent features. Features

PREVIEW

are measured by counting target characteristics which are collected during program execution in a simulator. They are presented as 28 metrics in four categories: parallelism – how well an algorithm scales in response to core count; compute – the diversity of instructions; memory – working memory footprint and entropy measurements which correspond to caching characteristics; and control – branching and program flow. The metrics collected are primarily used in the prediction of execution times, but since they are representative of structural characteristics of the underlying program and are free from architectural traits, they can be used in diversity analysis in benchmark suites, identifying program requirements which allows the automatic calculation of theoretical peak performance for a given device and examining the differences in kernels to show the phase-transitional properties of the application codes. We also discuss the design decisions made to collect AIWC features.

Finally, this work culminates in a methodology which uses AIWC features to train a random forest model capable of predicting accelerator execution times. We use this model to predict execution times for a set of 37 computational kernels running on 15 different devices representing a broad range of CPU, GPU and MIC architectures. The predictions are highly accurate, differing from the measured experimental run-times by an average of only 1.2%. A previously unencountered code can be instrumented using AIWC to allow performance prediction across the full range of modelled devices. The results suggest that this methodology supports the correct selection of the most appropriate device for a previously unencountered code, and is highly relevant to efficiently schedule codes to emerging heterogeneous supercomputing systems.

PREVIEW

Contents

Declaration

Acknowledgements

Abstract

1 Introduction **1**
1.1 Context . . . 2
1.2 Thesis Contributions . . . 3
1.3 Thesis Structure . . . 4
1.4 Publications . . . 4

2 Background Information and Related Work **5**
2.1 The Dwarf Taxonomy . . . 5
2.2 Accelerator Architectures in HPC . . . 6
2.3 The Open Compute Language (OpenCL) . . . 13
2.4 Benchmark Suites . . . 15
2.4.1 Rodinia . . . 16
2.4.2 OpenDwarfs . . . 17
2.4.3 SHOC . . . 17
2.5 Hardware Performance and Scaling . . . 18
2.6 OpenCL Performance . . . 20
2.6.1 Autotuning . . . 20
2.6.2 Phase-Shifting . . . 21
2.6.3 Measurements . . . 22
2.7 Offline Ahead-of-Time Analysis . . . 22
2.8 Program Diversity Analysis and Characterization . . . 23
2.8.1 Microarchitecture-Independent Workload Characterization . . . 23
2.8.2 Architecture Independent Workload Characterization . . . 24
2.8.3 Workload Characterization for Benchmark Diversity Analysis . . . 25
2.9 Performance Prediction for Heterogeneous Architectures . . . 26
2.10 Scheduling for Heterogeneous HPC Systems . . . 28

3 Extending the OpenDwarfs Benchmark Suite **30**
3.1 Extending the OpenDwarfs Benchmark Suite . . . 31
3.2 Experimental Setup . . . 33

PREVIEW

3.2.1 Hardware . . . 33
3.2.2 Software . . . 34
3.2.3 Measurements . . . 34
3.2.4 Problem Size . . . 35
3.2.5 kmeans . . . 35
3.2.6 lud, fft, srad, crc, nw . . . 36
3.2.7 csr . . . 36
3.2.8 dwt . . . 37
3.2.9 gem, nqueens, hmm, swat . . . 37
3.2.10 bfs, cfd, tdm . . . 38
3.2.11 Summary of Benchmark Settings . . . 38
3.3 Results . . . 39
3.3.1 Time . . . 40
3.3.2 Energy . . . 43
3.4 Discussion . . . 44

4 AIWC: OpenCL based Architecture Independent Workload Characterization 51
4.1 Metrics . . . 53
4.2 Implementation . . . 55
4.3 Demonstration . . . 58
4.4 Detailed Analysis of LU Decomposition Benchmark . . . 59
4.5 Use Case: AIWC analysis of OpenDwarf bioinformatics related benchmarks . . . 64
4.6 Usage and Limitations . . . 66
4.7 Summary . . . 70

5 Making Performance Predictions for Scheduling 71
5.1 Model Development . . . 72
5.1.1 Experimental Setup . . . 72
5.1.2 Constructing the Random Forest Performance Model . . . 73
5.1.3 Parameters for the Random Forest Performance Model . . . 76
5.1.4 Tuning the Random Forest Model . . . 76
5.2 Evaluation . . . 78
5.2.1 Predicting Kernel Execution Time . . . 78
5.2.2 Choosing The Optimal Accelerator for a Kernel . . . 80
5.3 Discussion . . . 82

6 Conclusions and Future Directions 84
6.1 Extended OpenDwarfs – EOD . . . 85
6.2 AIWC . . . 86
6.3 Performance Prediction . . . 87
6.4 Future Directions . . . 88
6.4.1 EOD . . . 88
6.4.2 AIWC . . . 89

PREVIEW

6.4.3 Performance Predictions 89
6.4.4 Finding holes in benchmarks: Evaluating the coverage and corresponding performance predictions for conventional vs synthetic benchmarking 90
6.4.5 AIWC for the Masses: Towards language-agnostic architecture-independent workload characterization 90
6.4.6 Examining the Characteristics of Scientific Codes in Supercomputing with AIWC 91
6.4.7 Guiding Device Specific Optimization using Architecture-Independent Metrics 91
6.4.8 Faster FPGA development with AIWC and the Predictive Model 91
6.5 Closing Remarks 92

Appendices **92**

A Time Results **95**

B Linear Model Fitting **103**

C Diversity Analysis **107**

D Abbreviations **111**

References **113**

PREVIEW

List of Figures

2.1 The percentage of accelerators in use and the contributions of cores found on systems with accelerators in the Top500 supercomputers over time. 11
2.2 Power efficiency (GFlops/Watt) of using accelerators in the Top500 supercomputers over time. 12

3.1 Kernel execution times for the medium problem size benchmarks on different accelerator devices. 46
3.2 Kernel execution times for the single sized benchmarks on various accelerator devices. 47
3.3 Kernel execution times for the **crc** benchmark on different hardware platforms. 48
3.4 Kernel execution times for the **kmeans** benchmark on different hardware platforms. 49
3.5 Execution energy required to perform EOD benchmarks, presented on a linear (a) and logarithmic scale (b) from left to right respectively, on the (**large** problem size) on the Intel i7-6700K and Nvidia GTX1080. 50

4.1 Selected AIWC metrics from each category over all kernels and 4 problem sizes. 60
4.2 A) and B) show the AIWC features of the `diagonal` and `internal` kernels of the LUD application over all problem sizes. 61
4.3 A) shows the AIWC features of the `perimeter` kernel of the LUD application over all problem sizes. B) shows the corresponding Local Memory Address Entropy for the `perimeter` kernel over the tiny problem size. 62
4.4 Architecture-Independent Workload Characterization features for selected bioinformatics benchmarks. 65
4.5 Architecture-Independent Workload Characterization features for the `hmm` bioinformatics benchmark. 66
4.6 EOD runtimes for **small** problem sized bioinformatics benchmarks. 67

5.1 Full coverage of min.node.size with fixed tuning parameters: num.trees = 300 and mtry = 30. 74
5.2 Full coverage of num.trees and mtry tuning parameters with min.node.size fixed at 9. 75
5.3 Prediction error across all benchmarks for models trained with varying numbers of kernels. 79
5.4 Predicted vs. measured execution time (in log(µs)) for all kernels. 79

PREVIEW

5.5 Error in predicted execution time for each kernel invocation over four problem sizes. 81
5.6 Mean measured kernel execution times compared against mean predicted kernel execution times to perform a selection of kernels on large problem sizes across 15 accelerator devices. The square indicates the mean measured time, and the diamond indicates the mean predicted time. 82

A.1 Kernel execution times for the **tiny** and **small** problem sizes of the `kmeans`, `lud`, `csr` and `dwt` benchmarks on different hardware platforms. 98
A.2 Kernel execution times for the **medium** and **large** problem sizes of the `kmeans`, `lud`, `csr` and `dwt` benchmarks on different hardware platforms. 99
A.3 Kernel execution times for the **tiny** and **small** problem sizes of the `fft`, `srad` and `nw` benchmarks on different hardware platforms. 100
A.4 Kernel execution times for the **medium** and **large** problem sizes of the `fft`, `srad` and `nw` benchmarks on different hardware platforms. 101
A.5 Single problem sized benchmarks of kernel execution times on different hardware platforms. 102

B.1 Predicted vs. measured execution time (in log(µs)) for all kernels and devices with the GLM model. 104
B.2 Distribution of prediction errors with the GLM model, presented in a log scale. 104
B.3 Prediction intervals for the `kmeansPoint` kernel on the GTX1080Ti GPU over four problem sizes. 105
B.4 Error in predicted execution time with a GLM model for each kernel invocation over four problem sizes. 106

C.1 Biplot Principal Components of AIWC metrics over all application kernels and all problem sizes. 108
C.2 The proportion of explained variance of each Principal Component. 109
C.3 The t-SNE with k-means cluster results to show Principal Components. 110

PREVIEW

List of Tables

2.1 The Berkeley Dwarfs and their limiting factors. 6

3.1 List of Extended OpenDwarfs Applications and their respective dwarfs. 32
3.2 Hardware utilized during the Extended OpenDwarfs Benchmark Suite evaluation. 33
3.3 The different problem sizes in the Extended OpenDwarfs adjusted by selecting the workload scale parameter (Φ). 38
3.4 Program Arguments for benchmarks in the Extended OpenDwarf Suite. 39

4.1 ISA-Independent Workload Characterization metrics. 53
4.2 Metrics collected by the AIWC tool ordered by type. 54
4.3 Overhead of the AIWC tool on the `fft` benchmark and the Intel i7-6700K CPU. 68

5.1 Optimal tuning parameters from the same starting location for all models omitting each individual kernel. 77

PREVIEW

Chapter 1

Introduction

Supercomputers are used in computationally intensive tasks and are a critical tool in current scientific research. They are essential in simulations for quantum mechanics, weather forecasting, climate research, oil and gas exploration and molecular modelling. However the largest supercomputers are requiring huge amounts of electricity to operate, for example, the current world's fastest supercomputer, Summit [1], requires 8.8 MW to power, which in terms of the average Australian home (13.7 kWh per day) could power 15500 homes. To reduce this large energy footprint supercomputers are becoming increasingly heterogeneous. At an individual node, there is a trend towards specialised hardware – known as accelerators – which can expedite the computation of codes from particular classes of scientific workloads. The use of accelerators for certain programs offers a shorter time to completion, and less energy expenditure when compared to a conventional CPU architecture. The next generation of supercomputers has been designed to incorporate a greater number of accelerators, and varying types within a node. For instance, the CAPI [2] and NVLINK [3] technologies included in the latest IBM POWER9 [4] processor offers a high-speed interconnect which allows the rapid movement of data between processor and accelerator – where Nvidia Graphical Processing Unit (GPU) use NVLINK, whereas other accelerator devices such as Altera Field-Programmable Gate Array (FPGA), Digital Signal Processors (DSPs), Intel Many-Integrated-Core (MIC) devices, and both Intel and AMD Central Processing Unit (CPU) and AMD GPU devices can utilise the CAPI interconnect. The support from hardware vendors for a greater mix of heterogeneous devices indicates this is the future direction of supercomputing. However, this development is recent, and as such the scheduling of workloads to the most suitable accelerator becomes an issue. Without addressing this the cost of exascale computing and the corresponding energy efficiency will be prohibitive.

This thesis will argue that certain characteristics of a scientific code, specifically around computation, memory, branching and parallelism, are independent of any particular device on which they may be finally executed and that the metrics used to quantify each of these characteristics can be collected during program execution on a simulator. In other words, provided they are collected over a representative workload, a graph traversal program maintains the characteristics of a graph traversal program regardless of problem size or on what platform it is run. Moreover, these metrics can be used to accurately predict the execution

PREVIEW

time on each accelerator in a heterogeneous system.

This thesis also presents a methodology to perform runtime predictions for a given code – provided the feature metrics are pre-generated – on any accelerator device. A benchmark suite is extended, a characterisation tool developed, and a model is generated to achieve the task. We believe this research will enable scheduling of codes to the most appropriate device to achieve better performance and utilization on the next-generation of supercomputers.

1.1 Context

Accelerators can be programmed in a variety of different languages – CUDA [5] for Nvidia GPUs, ROCm [6] for AMD devices, OpenMP [7] on Intel CPUs and the MIC. The Open Compute Language (OpenCL) [8] allows programs to be written once and run anywhere on a range of accelerators. A majority of accelerator vendors ship products with an OpenCL supported runtime, many of which will be components on the next-generation of supercomputing nodes. Programs in the OpenCL setting are structured into two parts, the host and the accelerator/device side. The developer is responsible for allocating and transferring memory between the host and the device. This requires programs to be structured with computationally intensive regions of code – known as kernels – to be identified and written in the OpenCL C kernel language. Kernels are viewed as indivisible functions, and as such, the nature of these kernels is fixed for all executions. Specifically, a kernel does not suffer from the phase-transitions that are common when looking at larger scientific codes. The composition of all kernels forms an accelerator agnostic implementation for full scientific applications.

A benefit of the fixed/static nature of OpenCL kernels is that the collection of the characteristics is also constant – the patterns of computation and communication do not change. Instrumentation of the execution of a kernel measures computation, memory, branching and parallelism metrics – these form the characteristics of a program and are largely unchanged between run and are independent of data set. To this end, we developed the Architecture Independent Workload Characterisation (AIWC) tool. This tool collects 28 metrics that indicate computation, memory, branching and parallelism characteristics on a per kernel basis. It simulates an OpenCL device using the Oclgrind [9] tool. The AIWC plugin analyses a kernel's execution trace, including the memory locations accessed and thread-states, to generates simple metrics that are representative of the kernel's behaviour. Metrics can be collected quickly since it is a multi-threaded simulator. AIWC features, are generated for each kernel invocation and can be embedded as a comment into the header of OpenCL kernel codes – either in plain-text source or in the Standard Portable Intermediate Representation (SPIR) [10] format.

There are few high-performance scientific computing benchmark suites which are capable of execution over a wide range of accelerators with portable performance and reproducible results. Additionally, until this work was undertaken, the available OpenCL benchmark suites

PREVIEW

were not rich enough to adequately characterise performance across the diverse range of applications or accelerator devices of interest. Thus this thesis presents an enhanced version of the OpenDwarfs OpenCL benchmark suite [11] – Extended OpenDwarfs (EOD) – which was developed with a strong focus on robustness, benchmark diversity, choice of problem size, and correctness of results.

LibSciBench [12] was added to EOD to provide high precision timers along with support for the collection of hardware events and energy usage information. Kernel execution times of all EOD benchmarks were collected on 15 different accelerator devices typical of HPC systems. Collection of these times occurs at a per kernel level along with instrumentation of other events common to the OpenCL setting, such as memory setup and timing data movement to accelerator devices. Total elapsed application execution time was also collected.

This thesis develops a predictive model, capable of accurately estimating the execution times of a kernel on any accelerator based solely on its AIWC metrics.

The AIWC tool was run and the features collected from all the kernels of EOD. These AIWC metrics were used as predictor variables into a random forest model, with the executions times of kernels used as the response variables for prediction. The accelerators examined in these predictions range from CPU, GPU and MIC, although, the methodology presented is expected to perform with other accelerators such as DSPs and FPGAs.

The final random forest model performs well and is capable of accurate predictions which on average differ from the measured experimental run-times by 1.2%, which correspond to absolute execution time mispredictions between 8µs and 1s depending on problem size. The model is capable of predicting execution times for specific devices based on the computational characteristics captured by the AIWC tool, which in turn, enables selection of the optimal accelerator device for a particular kernel.

PREVIEW

1.2 Thesis Contributions

The OpenDwarfs [11] benchmark suite is extended to include a greater range of scientific applications and over multiple problem sizes. Additionally, the extended suite incorporates a high precision timing library which is capable of measuring energy usage and execution times on any OpenCL device. The benchmark suite is run on a range of devices allowing a direct comparison to be made between these devices on a per-application basis.

Separately, an Architecture Independent Workload Characterisation (AIWC) tool is presented and shown to be capable of analysing kernels and extract a set of predefined features or characteristics. The benefits of AIWC include that it:

1) provides insights around the inclusion of an application into a benchmark suite via diversity analysis of the feature-space.

2) measures requirements in terms of FLOPs, memory movement and integer ops of any application kernel – which may indicate the theoretical peak performance for a given device.
3) can be used to examine the phase-transitional properties of application codes – for instance if the instruction mix changes over time in terms of the balance between floating-point and memory operations.

These metrics are used for creating a predictive model of the performance of OpenCL kernels on different hardware devices and settings. This model is then applied to predict the performance of an application for any given platform without additional instrumentation. This prediction adds information that can be incorporated into existing HPC schedulers and provides a negligible run-time overhead – codes are examined one time by the developer when instrumenting with AIWC and the AIWC features are embedded into the header of each kernel code to be evaluated by the predictive model at the time of scheduling.

1.3 Thesis Structure

Chapter 2 reviews the existing literature and current techniques used to schedule heterogeneous resources. Chapter 3 discusses the extensions added to the OpenDwarfs Benchmarking Suite in EOD. Chapter 4 highlights the construction, design decisions made and usage of the AIWC tool. Chapter 5 develops the predictive model and examines the accuracy of the final predictions. Chapter 6 discusses the conclusions of this thesis and the future work required for the predictive model to be incorporated into scheduling on future supercomputing systems.

1.4 Publications

- Johnston B and Milthorpe J "AIWC: OpenCL-based Architecture-Independent Workload Characterisation", LLVM-HPC workshop, SC18, Dallas, Texas, USA, 2018.
- Johnston B and Milthorpe J "Dwarfs on Accelerators: Enhancing OpenCL Benchmarking for Heterogeneous Computing Architectures", ICPP, Eugene, Oregon, USA, 2018.
- Johnston B, Falzon G and Milthorpe J "OpenCL Performance Prediction using Architecture-Independent Features", HPCS, Orleans, France, 2018.

PREVIEW

Chapter 2

Background Information and Related Work

The chapter presents background information, terminology and the related work drawn upon in the rest of this thesis. It provides a background for readers who might not be familiar with workload characterisation of programs, the associated performance metrics or composition of current HPC systems and how their performance is evaluated. The types of devices considered in this thesis and the benchmark suites examined can be broadly classified according to the Dwarf Taxonomy, as such, this Chapter begins with an introduction to the Dwarf Taxonomy. Next, we define accelerators and provide a brief survey regarding their use in supercomputing. The hardware-agnostic programming framework OpenCL is then presented. Finally, this section culminates in a discussion of benchmark suites, applications and where they are incorporated into the dwarf taxonomy.

PREVIEW

2.1 The Dwarf Taxonomy

In 2004, Phil Colella [13] identified seven motifs of numerical methods which he thought would be important for the next decade. Based on this style of analysis, The Berkeley Dwarf Taxonomy [14] was conceived to present the motifs commonplace in HPC. Initially performed by Asanovic et al. [15], the Dwarf Taxonomy claims that many applications in parallel computing share patterns of communication and computation. Applications with similar patterns are defined as being represented by a single dwarf. Asanovic et al. [15] present a total of 13 dwarfs, stating that whilst it was believed that more dwarfs could be added to this list in the future, all currently encountered scientific codes are classified as belonging to one or more of these dwarfs. For each of the 13 dwarfs the authors indicate the performance limit – in other words, whether the dwarf is compute bound, memory latency limited or memory bandwidth limited. The dwarfs and their limiting factors are presented in Table 2.1. Note, the **?** symbol indicates the performance limit was unknown at the time of publication – and to the best of our knowledge, none of these has been resolved since.

Dwarf	Performance Limit
Dense Linear Algebra	Compute
Sparse Linear Algebra	Memory Bandwidth and Compute
Spectral Methods	Memory Latency
N-Body Methods	Compute
Structured Grid	Memory Bandwidth
Unstructured Grid	Memory Latency
Map Reduce	**?**
Combinational Logic	Memory Bandwidth and Compute
Graph Traversal	Memory Latency
Dynamic Programming	Memory Latency
Backtrack and Branch and Bound	**?**
Graphical Methods	**?**
Finite State Machines	**?**

Table 2.1: The Berkeley Dwarfs and their limiting factors.

Implementations of the Dwarfs are discussed in the Benchmark Suites Section 2.4. The division of applications into dwarfs helps ensure diversity in the benchmarks which, in turn, provides a fair evaluation of scientific codes on HPC nodes.

2.2 Accelerator Architectures in HPC

We will use the term "accelerators" to refer to any form of hardware specialized to a particular pattern of computation; Thus, specialized hardware may accelerate a given application code according to that codes characteristics. Accelerators commonly include GPU, FPGA, DSP and MIC devices. We define accelerators to include all compute devices, including CPUs since their architecture is well suited to accelerate the computation of specific dwarfs. Applications depicted by dwarfs in The Dwarf Taxonomy, Section 2.1, offer a wide range of characteristics realistic to scientific computing applications.

Lee et al. [16] evaluate the validity of reported speedup results between CPU and GPU devices over five benchmarks while considering optimizations which impact the characteristics of the codes. They find optimizations such as multithreading, cache blocking, and reorganization of memory accesses strongly affect CPU devices whilst minimizing global synchronization and using locally shared buffers benefit GPU devices. Two of their benchmarks are better suited for CPUs and suggest structural characteristics of applications impact on device performance. By extension, we expect that the performance of an application which is representative of a given dwarf will depend on the execution device and the structural characteristics which define that dwarf, in other words, certain applications are better suited to specific types of accelerator. The remainder of this section will present and describe each type of accelerator, its history and its uses.

Central Processing Units (CPU) have additional circuitry for branch control logic, and gener-

PREVIEW

ally operate at a high frequency, ensuring this architecture is highly suited to sequential tasks or workloads with many divergent logical comparisons – corresponding to the finite-state machine, combinational logic, dynamic programming and backtrack branch and bound dwarfs of the Berkeley Dwarf Taxonomy. Additionally, it is increasingly common to combine heterogeneous CPUs in a System-on-a-Chip (SoC). Comprised of two separate micro-architectures, SoCs have a high-performance CPU – faster base clock speed with additional hardware for branching – to support the irregular control and access behaviour of typical workloads; and a smaller CPU – commonly with a lower base-clock frequency but with many more cores and support for longer vector instructions – for the highly parallel workloads/tasks common in scientific computing.

The SW26010 and ARM big.LITTLE type processors are current examples of how CPUs are treated as accelerators to achieve performance on modern supercomputers. The SW26010 CPU deployed in the Sunway TaihuLight supercomputer contains high-performance cores known as Management Processing Elements (MPE), and low-powered Computer Processing Elements (CPE). The CPE is arranged in an 8x8 mesh of cores, supports only user mode, and each core sports a small 16 KB L1 instruction cache and 64 KB scratch memory. Both MPE and CPE are of 64-bit Reduced Instruction-Set Computers (RISC) and support 256-bit vector instructions. This configuration shows the intent of the architecture, that the smaller CPEs need be used effectively to achieve good performance [17]. In other words, the host or primary core contributes only a small part of the maximum theoretical FLOPs on modern heterogeneous supercomputers.

ARM processors with big.LITTLE and dynamIQ configurations have been proposed to meet the power needs of exascale supercomputers [18]–[21]. big.LITTLE is a heterogeneous configuration of CPU cores on the same die and memory regions. The big cores have higher clock frequencies and are generally more powerful than the LITTLE cores, which creates a multi-core processor that suits a dynamic workload more than clock scaling. Tasks can migrate to the most appropriate core, and unused cores can be powered down. CPUs can be considered accelerators since many heterogeneous configurations including the SW26010 and big.LITTLE devices have side cores, which, with careful work scheduling, can accelerate workloads and achieve high FLOPs. Additionally, the heterogeneous configuration of side cores on modern CPUs presents a similar set of work-scheduling problems, that occur on other accelerators, primarily, these cores need to be given the appropriate work to ensure good system performance.

Graphics Processing Units (GPU) were originally designed to accelerate manipulating computer graphics and image processing, which is achieved by having circuit designs to apply the same operation to many values at once. This highly parallel structure makes them suitable for applications which involve processing large blocks of data. Many of the dwarfs of scientific computation are directly suited to GPUs for acceleration, including dense [22][23] and sparse linear algebra and N-Body methods. There has been an active effort to migrate applications from less suited dwarfs, such as spectral methods [24], structured grids [25] and

PREVIEW

graph traversal [26] for GPU acceleration. Efforts are primarily algorithmic, such as reordering of operations and the padding of shared memory, and have been used with varying success on GPU architectures [27]. Avoiding bank-conflicts and non-coalesced memory accesses thus increasing the use of private and shared memory are critical to the performance of these dwarfs on GPUs. They are the most common type of accelerator in supercomputer systems. The recent adoption of the Nvidia Volta GV100 GPU as the primary accelerator into the Summit and Sierra supercomputers [28] is attributed to its performance [29] and energy efficiency [30] on workloads fundamental to scientific computing.

Many Integrated Core (MIC) architectures are an Intel Corporation specific accelerator. Xeon Phi formerly known as Knights Landing (KNL) is the last series of the MIC accelerators and was discontinued in July 2018. It is significantly different to a GPU, it relies heavily on Single Instruction Multiple Data (SIMD) parallelism as opposed to the Single Instruction Multiple Thread (SIMT) needed for GPUs. It has many low-frequency in-order cores sharing the same bus and each core is based on conventional CPU x86 architectures. There are 72 cores with a layout based on a 2D mesh topology – comprised of 38 tiles, each tile features two CPU cores, and each core contains two Vector Processing Units (VPU). [31]; four cores are reserved for host-side system control and orchestration of work to the other cores. A 2D cache-coherent interconnect between tiles is included provide high-bandwidth pathways to match the memory access patterns on the core and mesh layout – cores on the same tile have a shared 1 MB L2 cache. Each core supports 512-bit vector instruction to utilize a large amount of SIMD parallelism. Dwarfs such as Dense and Sparse Linear Algebra are high-intensity and throughput-oriented workloads suited to the Xeon Phi accelerator [32]. The Xeon Phi is the primary accelerator in the Trinity [33] and Cori [34] supercomputer systems – currently in the top 10 of the Top500.

Field-Programmable Gate Arrays (FPGA) are accelerators which allow the physical hardware to be reconfigured for any specific task. They are composed of a high number of logic-gates organised into logic-blocks with fast I/O rates and bi-directional communication between them. FPGAs are suitable for workloads which require simple operations on very large amounts of data with a fast I/O transfer. Specifically, they are well suited to accelerating applications from spectral methods dwarf, specifically stream/filter processing on temporal data, and the combinational logic dwarf, which exploits bit-level parallelism to achieve high throughput. An example of the combinational logic dwarf is in the computing of checksums which is commonly required for network processing and ensuring data archive integrity. The configurablity of these devices may make them well suited to the characteristics of many dwarfs, however, the compilation or configuring the hardware for an application takes many orders of magnitude longer than any of the other examined accelerator architectures. Akram et al.[35] present a prototype FPGA supercomputer comprised of 5 compute nodes, each with an ARM CPU and Xilinx 7 FPGA. The benchmark application was of a Finite Impulse Response Filter – an application typical of the Spectral Methods dwarf – and presents 8.5× performance improvement over direct computation on the ARM CPU alone. Unfortunately, energy efficiency or a comparison between GPU accelerators is not presented. Fujita et al. [36] present a

PREVIEW

comparison between a P100 GPU and BittWare A10PL4 FPGA over an Authentic Radiation Transfer scientific application and show that the performance is comparable, however, an energy efficiency comparison between these two accelerators is not presented. Given the increasing need for high-throughput devices from applications in combinational logic and other dwarfs, FPGA devices are likely to be included in future HPC systems.

An integrated circuit designed solely for a specific task is known as an Application-Specific Integrated Circuit (ASIC). In this regard, they are akin to FPGAs without the ability to be reconfigured. They have been used to accelerate the hashing workloads from the combinational logic dwarf for bitcoin mining tasks. Google's Tensor Processing Units (TPU) are another example of ASICs, and support the TensorFlow [37] framework. TPUs perform convolutions for Machine Learning applications, which require many large matrix operations and are encapsulated by both the dense and sparse linear algebra dwarfs [38].

Digital Signal Processors (DSP) have their origins in audio processing – specifically in telephone exchanges and more recently in mobile phones – where streams of data are constantly arriving and an identical operation must be applied to each element. Audio compression and temporal filtering are examples of the Spectral Methods dwarf and are best suited to the DSP architecture. DSP cores operate on a separate clock to the host CPU and have circular memory buffers which allow a host device – using shared memory – to provide and remove data for processing without ever interrupting the DSP. Mitra et al. [39] evaluate a prototype nCore Brown-Dwarf system where each node contains an ARM Cortex-A15 host CPU, a single Texas Instruments Keystone II DSP and two Keystone I DSPs. They compare the performance and energy-efficiency of dense matrix multiplication and a real-world scientific code for biostructure based drug design against conventional x86 based HPC systems with attached accelerators. They show a Brown-Dwarf node is competitive with contemporary systems for memory-bound computations and show the C66x multi-core DSP is capable of running floating-point intensive HPC application codes.

Reagen et al. [40] consider the energy efficiency of accelerators by exploring properties of workloads and how these properties affect the shape of the large design space of hardware accelerators. They examine implementations of five benchmarks in terms of energy and explore the architectural parameters by sweeping through four configurable directives – loop unrolling, array partitioning, pipelining and Multiplier Stages. Their work provides insight around which workloads benefit most from acceleration.

Research around the suitability of ARM CPUs for HPC systems is highly active, with comparisons against the conventional Intel and AMD CPUs being made and the potential strengths of ARM systems when striving for energy efficiency [41][42][43]. Isambard [44] and Astra [45] systems use the Cavium ThunderX2 CPU accelerator, where each ThunderX2 accelerator consists of 32 high-end ARM cores operating at 2.1 GHz [46]. Separately, Fujitsu proposes using ARMv8-A cores for the Post-K supercomputer [47]. In a similar layout to the ThunderX2 the FX100 is a Scalable Many Core (SMaC) with the memory model – Core Memory Group – and core configuration – Compute Engine – also in a grid layout.

PREVIEW

Currently, only 25 of the Top500 systems are based on ARM technologies, but these experimental systems may indicate the way forward for exascale supercomputing. The most compelling reason for this transition to ARM is improved energy efficiency. ARM processors were originally targeted for embedded and mobile computing markets, where energy efficiency is a major constraint and may explain that while time-to-completion of tasks is higher versus conventional x86 architectures, the energy usage is much lower. Simula et al. [48] evaluate ARM processors against conventional x86 processors on real-time cortical simulations and consider the energy and interconnect scaling over distributed systems. They show joules per synaptic event on a network of ARM-based Jetson systems use 3× less energy than the Intel solution, whilst being 5× slower. The benchmark identifies an interesting bottleneck on current HPC x86 based systems: as the problem sizes grow larger more nodes and a larger network is required, thus, it is the lack of a low-latency, energy-efficient interconnect that is the primary concern. However, since ARM-based HPC systems can be populated more densely and offer a lower baseline energy profile, it is an architecture better suited to bio-inspired artificial intelligence applications and scientific investigations of the cognitive functions of the brain.

A major motivation for the increasing use of heterogeneous architectures is to reduce energy use; indeed, without significant improvements in energy efficiency, the cost of exascale computing will be prohibitive [49]. The diversity of accelerators in this space is best shown in a survey of accelerator usage and energy consumption in the worlds leading supercomputers. The complete results from the TOP500 and Green500 lists [50] were examined, over consecutive years from 2012 to 2018. The starting year was selected as 2012 because it was the first occurrence in the TOP500 spreadsheets to provide both accelerator name and accelerator core count information. Each dataset was taken from the June editions of the yearly listings.

Figure 2.1 shows a steady increase in the use of accelerators in supercomputers depicted as the solid purple line. This is presented as a percentage of the number of systems using accelerators in the TOP500 divided by 500 – the total number of systems listed in the TOP500 every year. In 2012 and 2013 11% of systems in the TOP500 used accelerators, this increased by roughly 2% per year. As of 2018, 22% of the TOP500 use accelerators. Note, from 2016 the Sunway TaihuLight system was introduced and is in the top 10, however, due to the reliance on the CPE side-core to achieve the FLOPs for its rank, the data was adjusted to be listed as containing an accelerator [17]. Also shown in Figure 2.1 is the average percentage of cores in the TOP500 every year dedicated to accelerators, presented as the dashed blue line. This measure indicates how much of the TOP500 compute is dependent on the accelerator – for systems that contain accelerators. The rationale for this metric is that systems in the TOP500 which use accelerators are not only accelerator-based systems – they contain conventional x86 CPU architectures as a host-side device which mirror the non-accelerator HPC systems, the teal line indicates what percentage of compute resources are attributed to the accelerator. Unsurprisingly, every year from 2012 to 2018, we see that a greater contribution of system resources – cores – are dedicated for accelerator devices and fewer resources for systems with accelerators are provided for the host. In 2012, 63% of supercomputer cores were located on the accelerator, by 2013 it jumped to 76%, this increased on average by 1.5% per year to 85%

PREVIEW

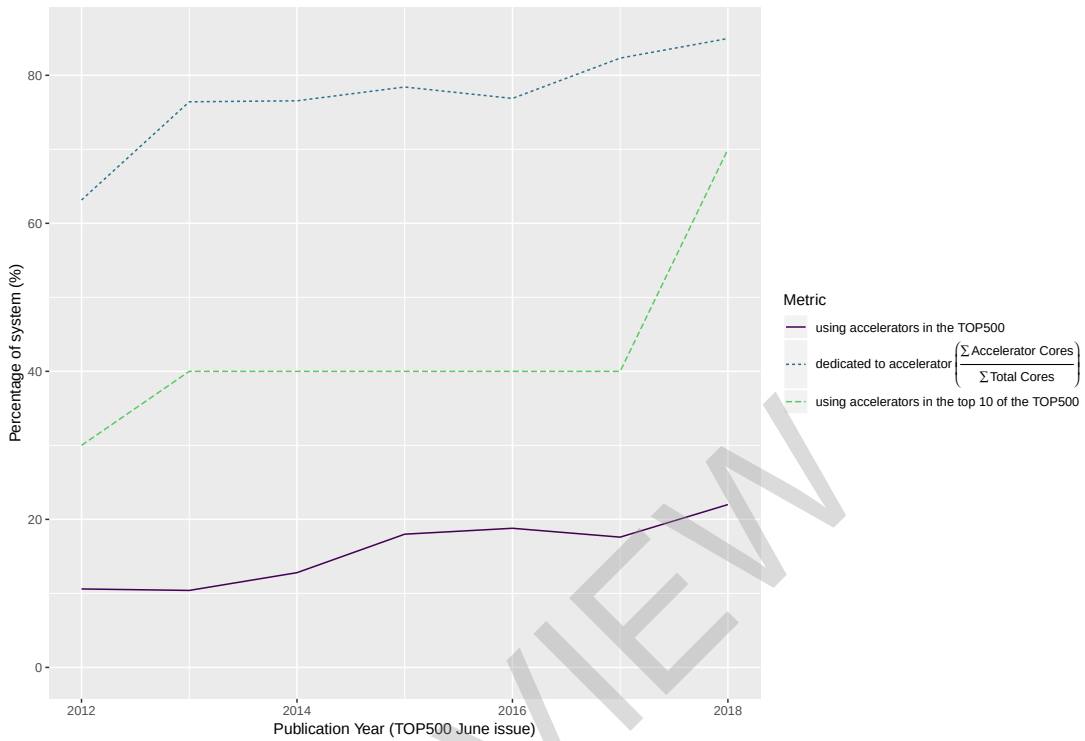

Figure 2.1: The percentage of accelerators in use and the contributions of cores found on systems with accelerators in the Top500 supercomputers over time.

PREVIEW

of compute cores being accelerator based in 2018.

A closer inspection of the top 10 of the TOP500 systems over the same time period is presented as the green long dashed line in Figure 2.1 and shows a greater dependence on accelerators and a corresponding increase in heterogeneity. In 2012, three out of the top 10 supercomputers used accelerators to secure a position. From 2013 to 2017, the use of accelerators in these systems was consistently at 40% however in 2018 it jumped to 70%. Since the use of accelerators in the top 10 is much higher than in the rest of the TOP500 (purple line), we can conclude that the use of accelerators gives an edge to the ranking of these systems. The general trend of increased use of accelerators throughout all of the TOP500 continues to increase and reinforces the importance of accelerators in this space. Another benefit from the increasing dependence on a heterogeneous mix of accelerator devices is improved energy efficiency on these systems.

Figure 2.2 presents a comparison of the energy efficiency – the rate of computation that can be delivered by a computer for every watt of power consumed – in terms of billions of floating point operations per second per watt, of supercomputers which use accelerators, presented as the solid purple line, and systems which do not use accelerators – shown as the dashed teal line. Generally, we see that the mean energy efficiency of all systems improves over time. However, it is apparent that the use of accelerators in supercomputers has always offered better energy efficiency than using conventional x86 architectures as the primary

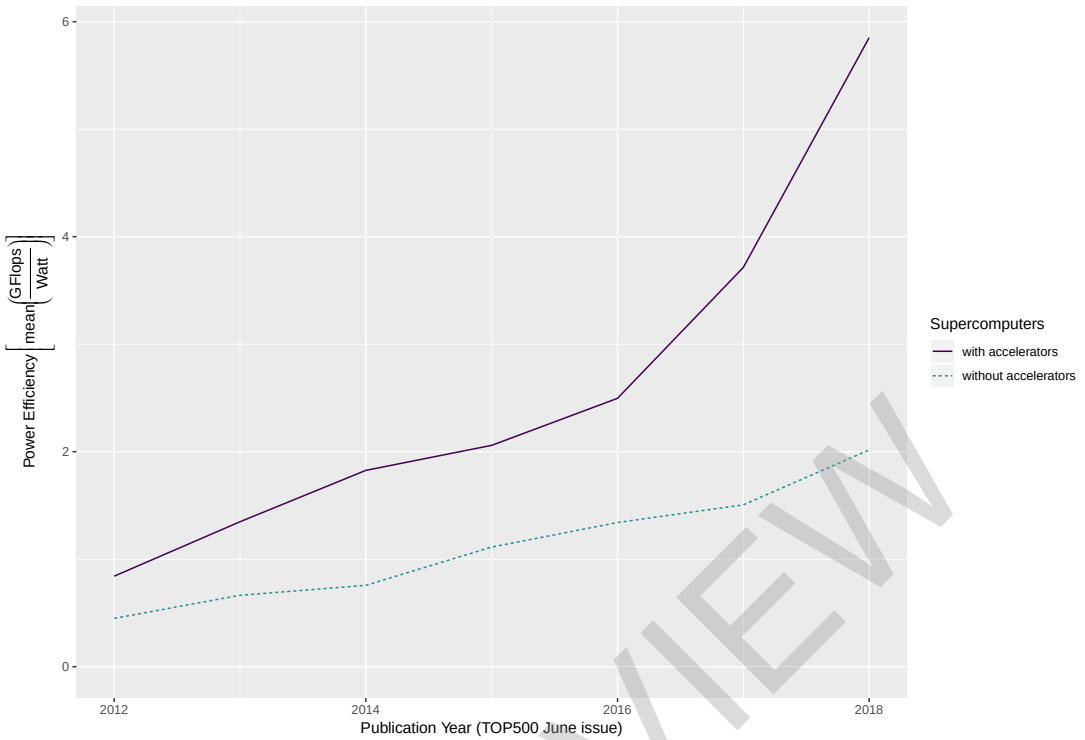

Figure 2.2: Power efficiency (GFlops/Watt) of using accelerators in the Top500 supercomputers over time.

means of computation. Systems without accelerators had a mean energy efficiency of 500 MFlops/Watt in 2012 and have increased on average by 200 MFlops/Watt every year, in 2018 these systems achieved 2 GFlops/Watt. These results are modest when compared to the gains in efficiency when using accelerators in supercomputing systems. In contrast, in 2012 the mean energy efficiency of supercomputers with accelerators was 900 MFlops/Watt and reached 5.9 GFlops/Watt in 2018, growing non-linearly by 750 MFlops/Watt per year. The efficiency of systems using accelerators is improving faster than supercomputers which rely on homogeneous CPU architectures.

Similar efficiencies have also been shown in the most energy efficient supercomputing list – the Green500 – where from June 2016 to June 2017, the average energy efficiency of the top 10 of the Green500 supercomputers rose by 2.3x, from 4.8 to 11.1 gigaflops per watt [50]. For many systems, this was made possible by highly energy-efficient Nvidia Tesla P100 GPUs. In addition to GPUs, future HPC architectures are also likely to include nodes with FPGA, DSP, ASIC and MIC components. A single node may be heterogeneous, containing multiple different computing devices; moreover, an HPC system may offer nodes of different types. For example, the Cori system at Lawrence Berkeley National Laboratory comprises 2,388 Cray XC40 nodes with Intel Haswell CPUs, and 9,688 Intel Xeon Phi nodes [51]. The Summit supercomputer at Oak Ridge National Laboratory is based on the IBM Power9 CPU, which includes both NVLINK [52], a high bandwidth interconnect between Nvidia GPUs; and CAPI,

PREVIEW

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.